SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – TRANSFORMER MECHANICS WORKSHEET

Course Code / Title	CSA6506 – Generative AI and Large Language Models
CO Assessed	CO1 – Precision (BL1, BL2)
Assessment	Assessment Tool 3 – Transformer Mechanics Worksheet
Weightage / Total Marks	40% of CIA | 100 Marks
CO1 Statement	Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models. (BL1, BL2)

Student Name	Madhavan S	Reg. No.	192425113
Section / Batch	CSE AI/2024	Date	15/10/2026

Instructions to Students

- Answer all questions. The worksheet carries 100 Marks.
- Tasks covered: tokenize a sentence; convert tokens to token IDs; explain embeddings; label a Transformer encoder; identify Query, Key and Value; explain which words receive higher attention; compare Transformer models; complete a Transformer pipeline; draw a block diagram.

Code of Conduct

I _____Madhavan S/192425113_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __ Madhavan S_____

SOLUTION: TRANSFORMER MECHANICS WORKSHEET

*Note: token splits, token IDs, embedding values and attention weights below are **illustrative** (they depend on the specific tokenizer and trained model); the mechanisms and formulas are exact.*

Question 1: Sentence Tokenization

Consider the sentence: “Transformers enable intelligent text generation.” Split the sentence into tokens using a suitable tokenizer. Explain how the tokenization may differ between a word-level tokenizer and a subword tokenizer.

Topic: Tokenization | Bloom’s level: Understand

SOLUTION

(a) Word-level tokenization (split on whitespace and punctuation)

`["Transformers", "enable", "intelligent", "text", "generation", "."]` → **6 tokens**

(b) Subword tokenization (BPE / WordPiece style)

`["Transform", "##ers", "enable", "intelligent", "text", "generation", "."]` → **7 tokens**
(WordPiece marks continuation pieces with `##`; byte-level BPE as in GPT uses a leading-space marker instead).

(c) How the two differ

Aspect	Word-level tokenizer	Subword tokenizer
Unit	Whole words (and punctuation)	Frequent words stay whole; rare or long words split into pieces (“Transformers” → “Transform” + “ers”)
Vocabulary size	Very large (every word form needs an entry)	Fixed and moderate (about 30k–100k pieces)
Unknown words	Out-of-vocabulary words collapse to `[UNK]`	Almost no `[UNK]`; unseen words are built from known pieces
Morphology	“enable”, “enabled”, “enabling” are unrelated entries	Shared stems and suffixes are reused, so related forms share information
Sequence length	Shorter	Slightly longer (7 vs 6 tokens here)

Conclusion: modern LLMs (BERT, GPT, T5) use subword tokenizers because they balance vocabulary size, coverage of rare words and sequence length.

Question 2: Token-to-ID Conversion

Given the token sequence ["AI", "models", "learn"] and the vocabulary mapping {"AI": 12, "models": 25, "learn": 8}, convert the tokens into their corresponding token IDs. What happens if a token is not present in the vocabulary?

Topic: Token IDs | Bloom’s level: Apply

SOLUTION

Each token is looked up in the vocabulary dictionary:

Token	Lookup	Token ID
"AI"	vocab["AI"]	12
"models"	vocab["models"]	25

Token	Lookup	Token ID
"learn"	vocab["learn"]	8

Result: `["AI", "models", "learn"] → [12, 25, 8]`

If a token is not in the vocabulary

- **Word-level / WordPiece:** the token is mapped to a special unknown token, e.g. `[UNK]` (say ID 0 or 1). Example: “quantum” (absent) → `[UNK]`, so `["AI", "quantum", "learn"] → [12, UNK_ID, 8]`. The model loses the word’s identity and meaning.
- **Subword BPE:** the word is split into smaller known pieces (e.g. “quant” + “um”); in byte-level BPE any text can be encoded as raw bytes, so true out-of-vocabulary cases do not occur.
- **Impact:** too many `[UNK]` tokens degrade understanding and generation quality, which is why subword tokenization is preferred.

Question 3: Understanding Embeddings

Explain how token IDs are converted into embedding vectors in a Transformer. Why are embedding vectors more useful for language processing than token IDs alone?

Topic: Token embeddings | Bloom’s level: Understand

SOLUTION

How IDs become vectors

1. The model owns a learnable **embedding matrix** `E` of shape `V × d` (V = vocabulary size, d = model dimension, e.g. 768).
2. A token ID `i` selects row `i` of `E`. This is a table lookup, mathematically the same as multiplying a one-hot vector by `E`.
3. A **positional encoding** (sinusoidal or learned) is added so the model knows word order: `x_i = E[id_i] + PE_i`.
4. `E` is trained by back-propagation together with the rest of the network, so the vectors encode meaning learned from data.

Toy example (d = 4): `12 ("AI") → [0.31, 0.64, −0.22, 0.18]`, `25 ("models") → [0.12, 0.58, −0.10, 0.30]`.

Why embeddings beat raw IDs

Property	Token ID	Embedding vector
Meaning	Arbitrary label; ID 25 is not “closer” to 26	Similar words get nearby vectors (cosine similarity)
Information	One integer	d dense numbers capturing syntax and semantics
Math operations	Arithmetic on IDs is meaningless	Dot products, addition and attention are meaningful
Learning	Fixed by vocabulary order	Learned and refined during training
Context	Same ID always the same	Becomes context-aware after attention layers

Question 4: Transformer Encoder Labelling

Draw a Transformer encoder block and label the following components in the correct order: multi-head self-attention, input embeddings with positional information, feed-forward neural network, residual connections, and layer normalization.

Topic: Transformer encoder architecture | Bloom's level: Apply

SOLUTION

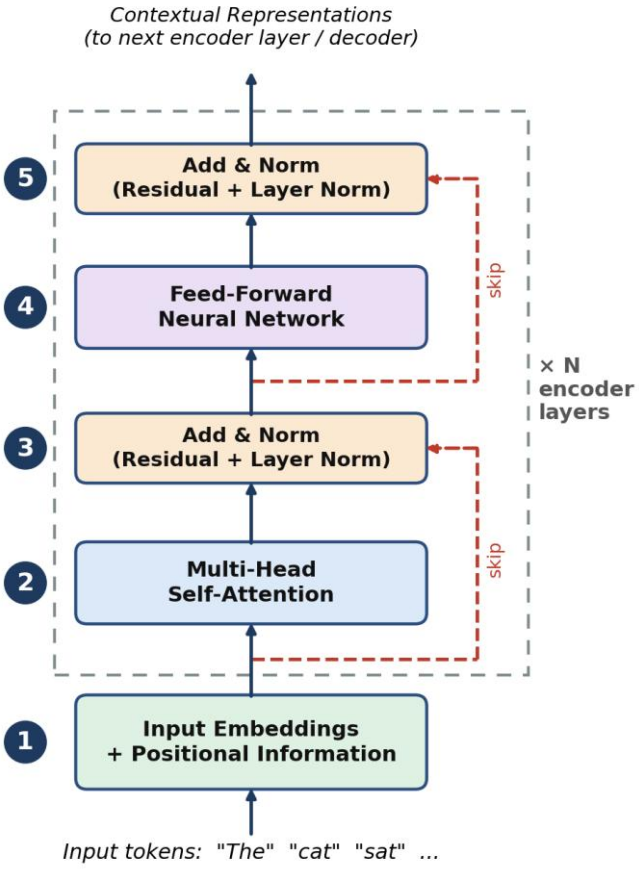


Figure 1: Transformer encoder block (data flows bottom to top)

Components in correct order

#	Component	Function
1	Input embeddings + positional information	Token IDs become vectors and word-order information is added.
2	Multi-head self-attention	Every token attends to all others using several heads in parallel to capture different relationships.
3	Residual connection + layer normalization (Add & Norm)	Input of the sub-layer is added to its output (eases gradient flow); layer norm stabilizes activations.
4	Feed-forward neural network	Position-wise two-layer MLP (d → 4d → d) that transforms each token independently.
5	Residual connection + layer normalization (Add & Norm)	Same Add & Norm applied around the feed-forward network.

The block from step 2 to step 5 is stacked **N times** (6 in the original Transformer, 12 in BERT-base).

Question 5: Query, Key, and Value Identification

In the sentence “The student submitted the assignment because it was complete,” explain how Query, Key, and Value vectors are generated from token representations. Describe their respective roles in determining relationships between words.

Topic: Query–Key–Value | Bloom’s level: Analyze

SOLUTION

Step 1: Token representations

Tokens: `The | student | submitted | the | assignment | because | it | was | complete` (9 tokens). After embedding and positional encoding we have a matrix `X` of shape `9 × d`; row x_i is the representation of token i.

Step 2: Generating Q, K, V

Three learned weight matrices project every token vector:

```
Q = X · W_Q      K = X · W_K      V = X · W_V
```

So for the token “it”: `q_it = x_it W_Q`, `k_it = x_it W_K`, `v_it = x_it W_V`. The same is done for all 9 tokens (and separately for every attention head).

Step 3: Roles

Vector	Role	In this sentence
Query (Q)	“What am I looking for?” The token asking for context.	q_it asks: which earlier noun do I refer to?
Key (K)	“What do I offer?” A label other tokens match against.	k_assignment and k_student advertise that they are candidate nouns.
Value (V)	“What information do I carry?” The content that is passed on.	v_assignment holds the meaning that flows into “it”.

Step 4: Computing relationships

```
score(i, j) = (q_i · k_j) / √d_k
α_ij      = softmax_j( score(i, j) )          (weights sum to 1)
output_i    = Σ_j  α_ij · v_j
```

The dot product q·k measures compatibility; division by $\sqrt{d_k}$ keeps values stable; softmax turns scores into attention weights; the weighted sum of Values gives a new, context-enriched representation. Because “was complete” describes something that can be completed (an assignment, not a student), q_it matches k_assignment most strongly, so “it” absorbs mostly the meaning of “assignment”.

Question 6: Attention Weight Analysis

Consider the sentence: “The cat chased the mouse because it moved quickly.” Which words might receive higher attention when processing the word “it”? Justify your answer and explain how attention weights help a Transformer capture contextual relationships.

Topic: Self-attention | Bloom’s level: Analyze

SOLUTION

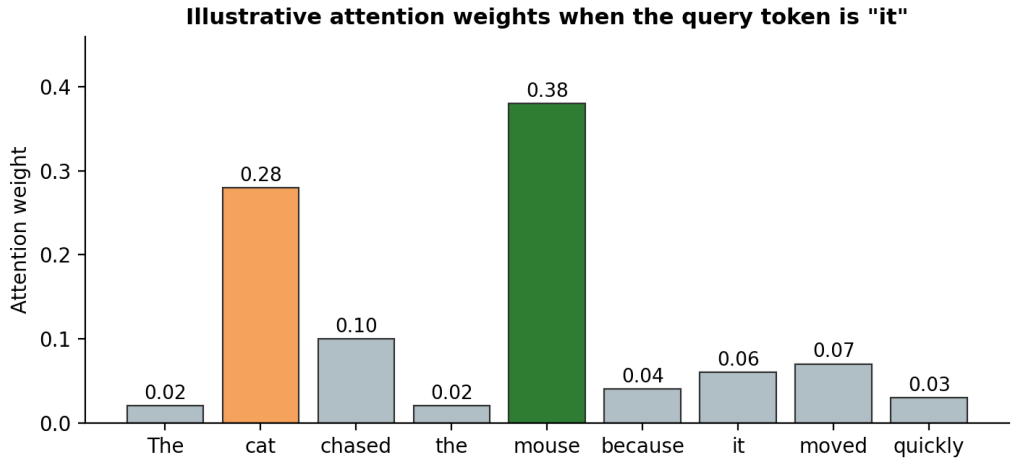


Figure 2: Illustrative attention distribution for the query token “it” (weights sum to 1.00)

Words receiving higher attention

- **“mouse” (highest):** “it moved quickly” most naturally describes the one being chased, which is escaping. The model has learned this from patterns of who “moves quickly” in such sentences.
- **“cat” (second):** a grammatical subject and the other candidate antecedent. The sentence is genuinely ambiguous (the cat could also have moved quickly), so attention is split between both nouns.
- **“chased” and “moved” (moderate):** the verbs provide the action roles that help decide who “it” is.
- **“the”, “because”, “quickly” (low):** function words and adverbs carry little referential information.

How attention weights capture context

- The weights decide how much of each word’s Value vector is mixed into the new representation of “it”, so “it” becomes a vector that effectively means “the mouse (or cat) that moved quickly”.
- This is **coreference resolution** learned without explicit rules, and it works regardless of distance between the words.
- Different heads specialise (coreference, syntax, nearby words), and the heads are combined for a rich contextual representation.
- The same word can therefore get different vectors in different sentences (contextual embeddings), unlike static embeddings.

Question 7: Comparing Transformer Models

Compare BERT, GPT, and T5 in terms of their attention approach, input-output behaviour, and typical applications. Identify which model would be most suitable for (a) text classification, (b) open-ended text generation, and (c) text summarization, with justification.

Topic: Transformer model comparison | Bloom's level: Analyze

SOLUTION

Feature	BERT	GPT	T5
Architecture	Encoder-only	Decoder-only	Encoder–decoder
Attention approach	Bidirectional self-attention: each token sees left and right context	Causal (masked) self-attention: each token sees only previous tokens	Bidirectional in encoder; causal in decoder plus cross-attention to encoder output
Pre-training	Masked language modelling (+ next-sentence prediction)	Next-token prediction	Span corruption (text-to-text denoising)
Input → output	Text → contextual vector per token (and `[CLS]` summary vector)	Prompt → continuation, one token at a time	Text with task prefix → new text (every task is text-to-text)
Typical applications	Classification, NER, question answering, semantic search	Chatbots, writing, code generation, few-shot tasks	Summarization, translation, question answering, paraphrasing

Best model for each task

Task	Model	Justification
(a) Text classification	BERT	Bidirectional attention gives a full-context sentence representation; a small classifier head on `[CLS]` is fine-tuned. No text generation is needed.
(b) Open-ended text generation	GPT	Autoregressive, causal design predicts the next token and matches how text is generated; it scales well and handles prompting.
(c) Text summarization	T5	The encoder reads the entire document bidirectionally, and the decoder generates the summary while cross-attending to it; the text-to-text format ("summarize: …") fits naturally.

Question 8: Completing the Transformer Pipeline

Complete the following Transformer processing pipeline by filling in the missing stages: Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token. Explain the function of each missing stage.

Topic: Transformer pipeline | Bloom's level: Apply

SOLUTION

Text Input → **Tokenization** → **Token ID Conversion** → Embedding and Positional Information → **Transformer Layers (Self-Attention + Feed-Forward)** → Output Probabilities → Predicted Token

Missing stage	Function
1. Tokenization	Splits raw text into tokens (words or subwords) using the tokenizer, e.g. "AI models learn" → ["AI", "models", "learn"].
2. Token ID Conversion	Maps each token to its integer ID via the vocabulary, e.g. [12, 25, 8], producing numbers the network can index.
3. Transformer Layers	Stacked blocks of multi-head self-attention (mixing context between tokens), feed-forward networks (per-token transformation), residual connections and layer normalization. They produce contextual representations.

Link to the next stage: the final hidden vectors pass through a linear output projection and **softmax** to give the output probabilities over the vocabulary; the predicted token is chosen by argmax or sampling.

Question 9: Transformer Block Diagram

Draw a block diagram illustrating how an input sentence is processed by a Transformer-based language model to generate an output. Include tokenization, token ID conversion, embeddings, attention layers, feed-forward layers, output projection, and token prediction.

Topic: Transformer architecture | Bloom's level: Create

SOLUTION

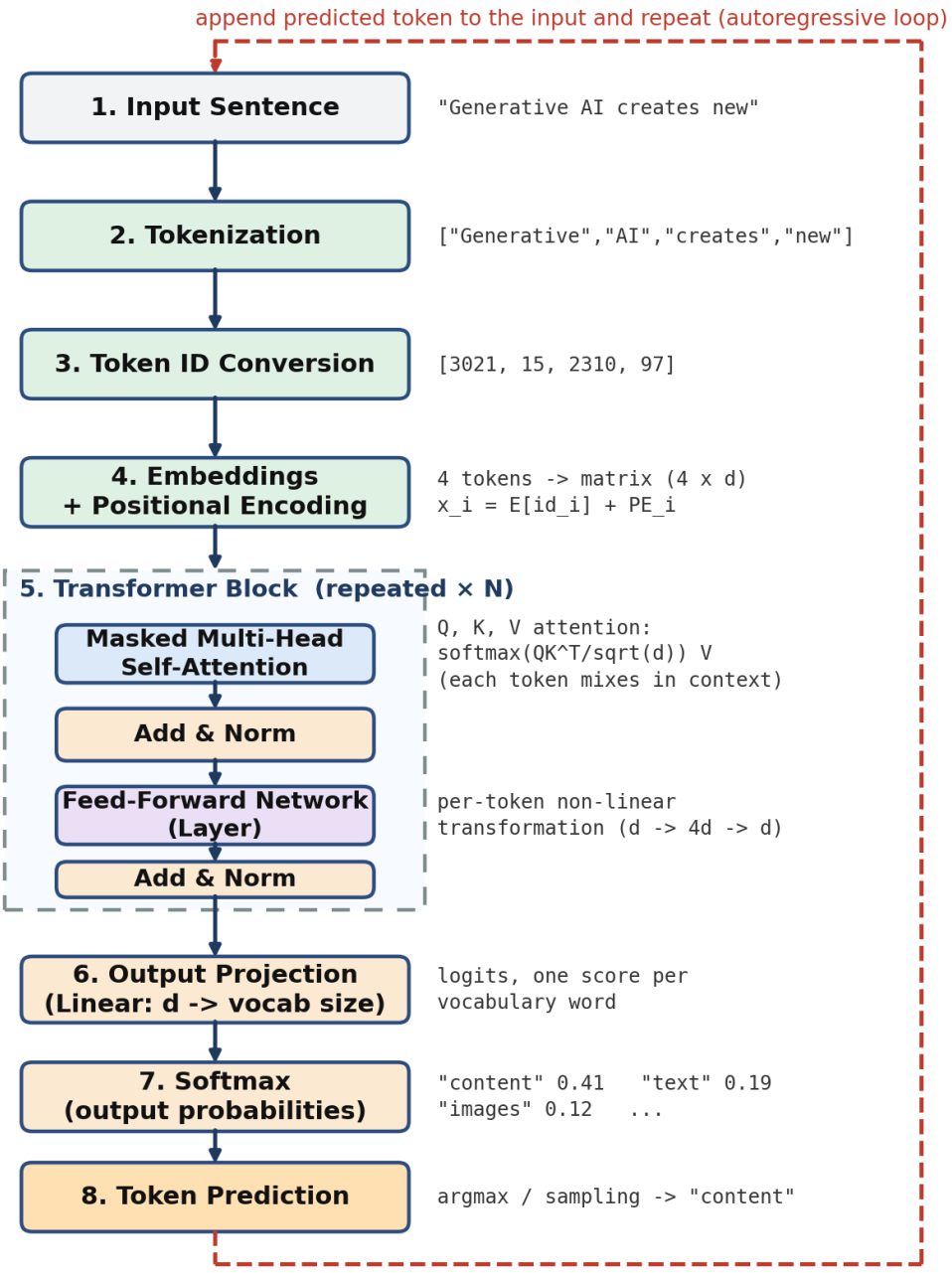


Figure 3: Block diagram of a Transformer-based (decoder-style) language model, with an example input

Reading the diagram: the input sentence is tokenized, converted to IDs and embedded with position information. N stacked Transformer blocks (masked multi-head attention, Add & Norm, feed-forward, Add & Norm) build contextual representations. The output projection gives a score for every vocabulary word, softmax converts scores to probabilities, and the predicted token is appended to the input so generation continues.

Question 10: Integrated Transformer Mechanics

A language model receives the sentence “Generative AI creates new content.” Trace the sentence through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. Explain how the attention mechanism helps the model understand relationships among the words.

Topic: Integrated Transformer pipeline | Bloom’s level: Analyze

SOLUTION

Step 1: Tokenization

`["Generative", "AI", "creates", "new", "content", "."]` (shown word-level for clarity; a real subword tokenizer might split “Generative” into pieces).

Step 2: Token ID conversion (illustrative vocabulary)

`[3021, 15, 2310, 97, 1184, 5]`

Step 3: Embedding generation

Each ID selects a row of the embedding matrix (toy d = 4; real models use 768 or more), then positional encoding is added:

Token	Embedding E[id] (toy)	Input vector x_i = E[id] + PE_i
Generative	[0.42, −0.13, 0.77, 0.05]	E + PE_0
AI	[0.31, 0.64, −0.22, 0.18]	E + PE_1
creates	[−0.51, 0.29, 0.40, 0.66]	E + PE_2
new	[0.12, −0.48, 0.09, 0.53]	E + PE_3
content	[0.70, 0.11, −0.35, 0.24]	E + PE_4
.	[0.01, 0.02, −0.03, 0.00]	E + PE_5

Step 4: Self-attention

Each x_i is projected to Q, K, V; scores `q·k/√d` are softmaxed into weights and used to average the Values. In a GPT-style language model a **causal mask** lets each token see only itself and earlier tokens. Attention row for the token “content” (illustrative):

Generative	AI	creates	new	content
0.08	0.17	0.30	0.33	0.12

“content” attends most to “new” (its modifier) and “creates” (the verb that takes it as object), and also to “AI” (the agent).

Step 5: Contextual representation

After the attention, Add & Norm, feed-forward network and further layers, each token has a **new vector that depends on its context**: “creates” now encodes “Generative AI creates…”, and “content” encodes “new content created by AI”.

Step 6: Output prediction

For the prefix “Generative AI creates new content”, the last token’s final vector goes through the output projection (d → vocabulary size) giving logits, then softmax gives probabilities, e.g. `"."` 0.62, `"for"` 0.12, `","` 0.08, `"in"` 0.05, others 0.13. The highest-probability token “.” is predicted (or sampled), and in training every position predicts its next token in parallel.

How attention helps the model understand word relationships

- It links words **directly**, regardless of distance: the subject “AI” connects to the verb “creates” and the object “content” in one step.
- It captures **roles**: who creates (AI), what is created (content), and what describes it (new, Generative).
- It runs for all positions **in parallel** and with **multiple heads**, so syntax, meaning and reference are learned simultaneously.
- Its output, contextual representations, is what makes accurate next-token prediction possible.

RUBRICS FOR CALCULATION

Criteria	Weight	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Mechanics Explanation	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Worked-Example Accuracy	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Mathematical Reasoning	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Diagram & Working Clarity	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Step Justification	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation

Mechanics Explanation (30%)	Worked-Example Accuracy (25%)	Mathematical Reasoning (20%)	Diagram & Working Clarity (15%)	Step Justification (10%)	Total (100)